

INFO0085-1 - Compilers

ARSANOV RAMZAN (S194447)

1 Lexical analysis

I had this already done because I did it last year. I did it 100% alone last year so I took it as it was. It handles everything from the submission platform.

2 Syntax and semantic analysis

2.1 Parser and AST

The parser uses Bison in c++ mode to build the AST. The base class ASTNode has a line, a col field and a virtual print method. Expr is the base for all expressions and has a "type" field for semantic annotations. The print method support both parse (-p) and semantic mode (-c) by only printing the type field when it is non-empty. I remember having a Shift/reduce conflict early in the project (hence the presence of the -Wcounterexamples flag in the Makefile). The initial grammar had a separate field_list and method_list rules in the class. Both of them start with OBJECT_IDENTIFIER. It was fixed by merging them into a single class_body rule. Other than that, I solved every other ambiguity with precedence.

2.2 Symbole table

symbole_table.cpp / symbole_table.hpp consist of Class_table, Class_info, Method_info, Field_info and Scope. Class_table is initialized with the Object class and its 6 methods. Class_info (and the others *_info classes) provides methods to travel through the inheritance chain. Scope implements the scoping for local variables. Each scope has its own variables map and a pointer to the enclosing scope.

2.3 Type checker

type_checker.cpp / type_checker.hpp implement a 4 pass algorithm:

- Collect class declarations, check for duplicates and redefinitions.
- Check inheritance (no parents, cycles).
- Collect field and method signatures, check if types exist, check override and verify the main class exists and is correct.
- Type check all expressions and fill the type field on every AST node.

3 Code generation

The code generator (ir.cpp/ir.hpp) iterates through the AST to produce LLVM intermediate representation in 6 steps:

- Define struct types for each class: a vtable pointer followed by inherited fields then the class's own fields.
- Define a vtable type for each class.
- Create the vtable instance for each class.
- Generate constructors that allocate memory, connect to the parent initializer, set the vtable pointer and initialize fields.
- Compile method bodies by recursively traversing expressions.
- Generate the main() entry point.

4 Limitations

No extensions were implemented.